Moderated Learning Management System

CS4433 Software Engineering: Individual Final Project Report

National University of Lesotho | Student: Thabelo Mapepesa | Student no.: 202321273 | Supervisor: Dr Morolong|
Repository: https://github.com/Kirito-36/Learning-Management-System.git (releases v1.0, v1.1)

1. Title Page, Abstract and Contents

Abstract. This report documents a Moderated Learning Management System (LMS) for a secondary school, built individually through the Waterfall model with V-Model verification. Administrators manage accounts and courses, lecturers set and mark assignments, Heads of Department (HoDs) moderate marks before release, and students submit work and read released marks. Conflicting stakeholder statements about mark visibility, late work and simplicity were resolved explicitly in the requirements. The result is a layered C++17 console application with SQLite persistence behind repository interfaces, Argon2id password hashing, a replaceable late-penalty Strategy, an explicit submission state machine and an append-only audit log. Verification used 61 automated GoogleTest cases with 91.3% line coverage of the business-logic layer. A change request (CR1, class-wide extensions) was delivered as v1.1.

Contents. 2 Introduction; 3 Project management; 4 Requirements; 5 Design; 6 Implementation; 7 Verification and validation; 8 Configuration management and quality assurance; 9 Deployment; 10 Maintenance (CR1); 11 Professional, legal and ethical issues; 12 Conclusion; 13 References; 14 Appendices (supplied as a separate file).

2. Introduction

A school needs one system for administrators, lecturers, Heads of Department and students, each limited to the actions their role allows. Submissions follow a defined life cycle and every mark change must record who, when and why. Payments, video, automated grading, plagiarism detection and web or mobile interfaces are out of scope, and the system must run on one school computer without reliable internet. The project followed the Waterfall model (Royce, 1970) with V-Model verification (Sommerville, 2016): requirements and design were completed and gated before any implementation, which suits fixed case-study requirements and a single developer. To offset Waterfall's late feedback, conflicts were resolved in week two and tests were written with the code. Tools are C++17, CMake, SQLite, libsodium, GoogleTest, GitHub Actions and cppcheck.

3. Project Management Plan

Feasibility. *Technical:* all tools are mature and free; libsodium avoids writing cryptography, and its interactive Argon2id settings (about 64 MiB) suit modest hardware. *Operational:* a short numbered menu per role mirrors the school's paper workflow. *Economic:* no licence or hardware cost beyond the existing computer. *Legal:* learner data falls under Lesotho's Data Protection Act, 2011, so the design minimises and protects it (section 11). *Schedule:* seven weeks fit only with a console interface and tests written alongside code, so a graphical interface was rejected. **Recommendation: proceed** with a layered console application on SQLite.

Schedule. The work breakdown follows the nine deliverables (planning, requirements, design, implementation, verification, QA and configuration management, deployment, maintenance, completion). Phases are finish-to-start, with gates after requirements (week 2), design (week 3) and verification (week 6). The critical path is requirements, design, implementation, verification, deployment and maintenance; milestones are the three gates and the v1.0 and v1.1 tags.

ID	Risk	L	I	Exp.	Mitigation
R1	Schedule overrun (one developer)	4	4	16	Fixed scope; Must requirements first
R2	Late rework from requirement conflicts	3	4	12	Resolve all conflicts in the SRS
R3	Security flaw exposes passwords or marks	2	5	10	Argon2id, bound SQL, role tests
R5	Data loss or corruption	2	5	10	Transactions, constraints, backups
R4	Slow on old computers	3	3	9	Indexes; 800-user test
R6	Build or library problems	3	3	9	CI build; install guide
R8	Scope creep	3	3	9	Out-of-scope list; change control
R7	Developer unavailable	2	4	8	Buffer in week seven

Table 1: Initial risk register (L = likelihood, I = impact, scale 1-5; owner: student, R8 supervisor). The final register is in the appendices.

4. Requirements Engineering

Stakeholders and conflicts. The principal, lecturers, HoDs, the student representative, the IT officer and the administrator were analysed. Table 2 lists the conflicts and ambiguities and their resolutions.

ID	Conflict or ambiguity	Resolution
C1	Principal: no mark seen before HoD check. Lecturer: marks visible once marked.	Principal's rule wins: students see only released marks (FR-13). Moderation is a quick approve-or-return step.
C2	HoD: zero after three late days. Students: late work not rejected.	Late work is always accepted and flagged; penalty applied at marking; zero after three days (FR-10).

ID	Conflict or ambiguity	Resolution
C3	Lecturer: extensions. HoD: strict penalties.	Individual extensions with a mandatory, audited reason (FR-07).
C4	Simple for teachers vs. secure and audited.	Security and audit are automatic; the visible interface stays a short role menu.
C5	Offline and fast vs. richer features.	Offline operation and 800-user speed take priority (NFR-01, NFR-03).
A1 - A3	What is “a day”, “the available mark”, “checked”?	Each started 24 h is a day; penalty is 10% of the maximum mark per day; the HoD approves, then releases separately.

Table 2: Conflicts (C) and ambiguities (A) with resolutions.

SRS (ISO/IEC/IEEE 29148 structure). Each requirement has a unique ID, MoSCoW priority and a test or inspection method. The functional requirements (Table 3) and measurable non-functional requirements (Table 4) are below.

ID	Functional requirement	Pri.
FR-01	Authenticate by username and password; one generic failure message	M
FR-02	Restrict every action to permitted roles	M
FR-03	Administrator creates accounts for all four roles	M
FR-04	Administrator creates departments, courses, lecturer allocations	M
FR-05	Administrator enrols students	M
FR-06	Lecturer creates assignments (deadline, maximum mark, attempt limit)	M
FR-07	Lecturer grants an individual extension, with reason	M
FR-08	Enrolled student submits an assignment	M
FR-09	Student resubmits before the deadline; earlier attempt kept as superseded	M
FR-10	Late work accepted, flagged; 10% of maximum per started day for 3 days, then zero	M
FR-11	Lecturer marks (0 to maximum) with mandatory written feedback	M
FR-12	HoD reviews all department marks: approve or return with comment	M
FR-13	Students see a mark only after HoD release	M
FR-14	Audit log of every mark change: who, when, old, new, reason	M
FR-15	Submission life cycle states enforced	M
FR-16	Student views courses, assignments, own submissions	M
FR-17	Administrator deactivates accounts	S
FR-18	User changes own password	S
FR-19	Administrator or HoD views audit history	S
FR-20	Class-wide extension (deferred; delivered by CR1)	W

Table 3: Functional requirements (M = Must, S = Should, W = Won’t this release).

ID	Non-functional requirement (measurable)	Pri.
NFR-01	Login and lookup within 2 s with 800 student accounts stored	M
NFR-02	Passwords stored only as salted Argon2id hashes	M
NFR-03	All core functions work offline	M
NFR-04	Data survives restart; failed multi-step operations leave no partial change	M
NFR-05	Each role’s functions within two menu selections; plain-language errors	S
NFR-06	100% of defined invalid inputs handled without terminating	M
NFR-07	At least 70% business-logic line coverage; no warnings at -Wall -Wextra -Wpedantic	S

Table 4: Non-functional requirements.

Use case model. The use case diagram (supplied in the appendices file because of space) covers four actors; the HoD is a specialisation of Lecturer. “Submit assignment” includes “Log in”; “Resubmit” and “Handle late submission” extend it; “Mark submission” includes “Apply late penalty”; “Request changes” extends “Moderate marks”. UC-01 is specified below; UC-02 (Mark submission) and UC-03 (Moderate and release marks) are fully dressed in the appendices file.

UC-01 Submit assignment	
Actor and preconditions	Student, logged in, active and enrolled in the course; assignment exists
Main flow	Student selects assignment and gives file path; system checks enrolment, computes the effective deadline (due date or latest extension), evaluates lateness, stores a new attempt with status Submitted and the late flag in one transaction, writes an audit entry and confirms
Alternative flows	Late: accepted, flagged, penalty or zero warning shown. Resubmission before the deadline: earlier attempt becomes Superseded
Exception flows	Not enrolled; attempt limit reached; resubmission after the deadline or after marking; invalid path; database error (rollback)
Postconditions	New attempt stored and one audit entry, or nothing changed
Business rules	Late first attempts never rejected; 10% per started day up to 3 days then zero (applied at marking); extension replaces the deadline

Table 5: Use case UC-01.

5. Software Design Document

Architecture. I chose a **layered architecture**: user interface, application services, domain logic, data access and a small security component, with downward dependencies only. SQL appears only in the data layer, and the domain depends on

interfaces for hashing, time and storage. The layered style separates business rules from persistence, makes services testable with fake repositories, lets a policy change touch one class, and yields one deployable unit. I **rejected microservices** (network communication and distributed deployment buy nothing for one school computer with unreliable internet) and **MVC as the primary style** (it suits interactive views over shared models, not this console request-and-response flow), although services act as controllers and repositories as models.

Class design and patterns. Figure 1 shows the class diagram. `User` is abstract, with `Administrator`, `Lecturer`, `HeadOfDepartment` (a Lecturer) and `Student`, so role behaviour is polymorphic. The **Strategy** pattern isolates the late-penalty rule in `ILatePenaltyStrategy`, implemented by `DailyLatePenaltyStrategy`, so a new policy changes no service. The **Repository** pattern gives each aggregate an interface with a SQLite implementation, confining SQL to one layer and allowing fakes in tests (Gamma et al., 1994; Fowler, 2002). Supporting abstractions are a unit of work for atomic updates and injected clock and hasher interfaces. The design follows SOLID: focused service classes (single responsibility), new penalties without editing marking code (open/closed), substitutable `User` subclasses (Liskov), small per-aggregate interfaces (interface segregation) and services that depend on interfaces, not SQLite or libsodium (dependency inversion). Implementation refined the Phase 3 design: the missing HoD class and audit columns (old value, new value, reason) were added, use-case logic moved from entities to services, and role-specific database constraints were introduced.

Figure 1: Class diagram (Phase 3)

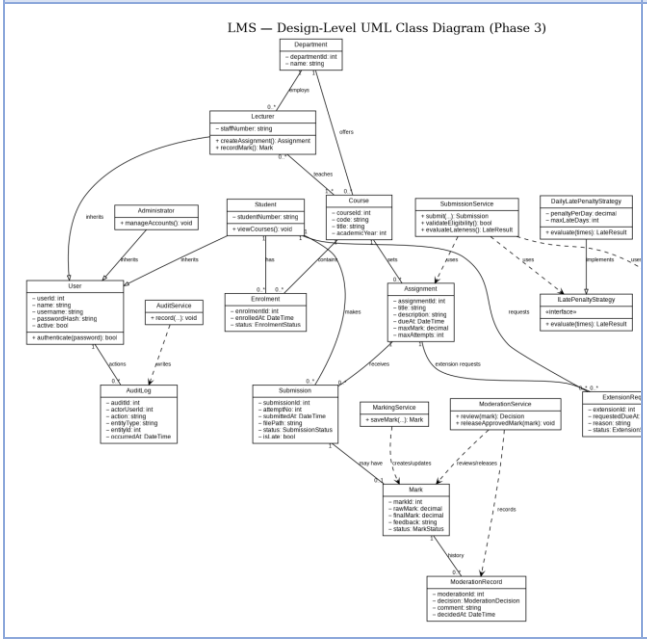
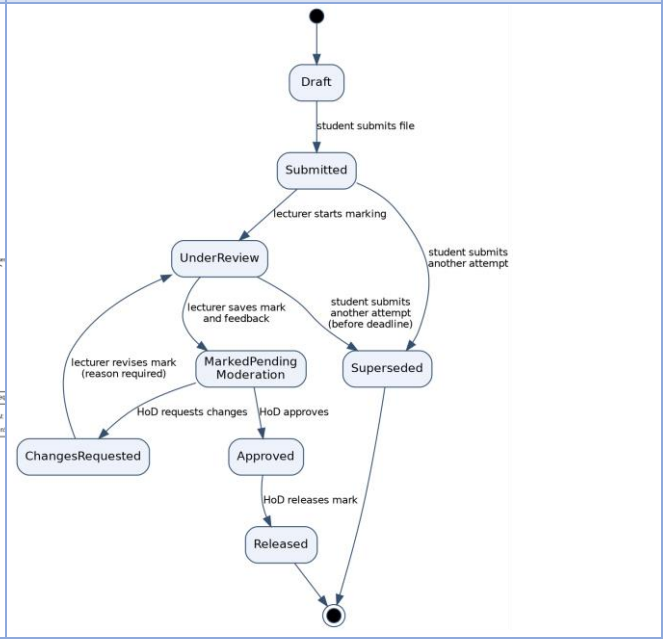


Figure 2: Submission state machine



Behaviour. The state machine (Figure 2) runs Submitted, Under review, Marked pending moderation, then Approved or Changes requested (back to Under review), then Released; a resubmission before the deadline moves the earlier attempt to Superseded. A mark is never visible merely because a lecturer entered it. The activity diagram with swimlanes and the two sequence diagrams (Submit assignment with late and resubmission paths; Moderate and release marks) from Phase 3 are in the appendices file.

Database. Eleven tables in Third Normal Form: departments, users, courses, course_lecturers, enrolments, assignments, submissions, extensions, marks, moderation_records and audit_log. Each fact is stored once and referenced by key; for example, moderation decisions have their own table because a mark can be reviewed repeatedly. The four user classes map to one `users` table (single-table inheritance with a role column and role-specific CHECK constraints), chosen over table-per-class because the roles share almost all attributes; submissions store one row per attempt so history is kept; triggers make `audit_log` append-only. The ERD, data dictionary and full SQL schema are in the appendices file and `sql/schema.sql`.

6. Implementation

The code follows the architecture: `src/domain`, `src/data`, `src/security`, `src/services` and `src/ui` (about 2,700 lines excluding the interface, plus 745 lines of tests), built with CMake under a documented coding standard (naming, no raw `new`/`delete`, SQL only in the data layer).

Key decisions. *Role-based access:* every service checks the role permission and then the relationship (the lecturer teaches the course, the student is enrolled, the HoD heads the department). *Late penalty:* late days are elapsed seconds divided by 86,400, rounded up; beyond three days the mark is zero, otherwise 0.10 per day of the maximum mark is deducted, floored at zero; it is computed at marking from the stored submission time and the effective deadline, so later extensions still give the right result. *Workflow:* first attempts are always accepted; resubmission needs an unmarked earlier attempt, time before the deadline and attempts remaining; marking requires feedback and, when changing a mark, a reason; students see only released marks. *Errors:* every multi-row use case runs in a unit of work that rolls back unless committed,

and service errors derive from one exception type caught in the console, so bad input or a database fault never crashes the program.

Security and persistence. Passwords use libsodium's `crypto_pwhash_str` (Argon2id, random salt embedded in the hash string), which resists GPU attacks, is recommended by OWASP and standardised in RFC 9106 (Biryukov et al., 2021), and avoids hand-written cryptography. Login returns one generic message for every failure and verifies a decoy hash for unknown users to hide timing differences. All SQL uses bound parameters; injection strings are tested. Foreign keys, CHECK constraints and audit triggers protect integrity. SQLite keeps all data in one file that survives restarts, and each change and its audit entry are committed together.

7. Verification and Validation

Strategy. Following the V-Model, unit tests verify the detailed design, integration tests the architecture (services on a real in-memory SQLite database), system tests the use cases including exception flows, and acceptance tests the stakeholder statements. Entry criterion for each level: the previous level passes; exit: all tests pass, every Must requirement exercised and no open critical defect. *Equivalence partitioning* split marks (below zero, valid, above maximum), lateness (on time, within three days, beyond) and roles. *Boundary value analysis* tested the penalty at deadline minus one second, the deadline, plus one second, plus 24 hours (one day), plus 24 hours one second (two days), plus 72 hours (30%) and plus 72 hours one second (zero), and also mark limits, attempt limits, name and password lengths and calendar dates.

Results. 61 GoogleTest cases in 15 suites all pass: 29 unit tests (penalty 13, state machine 6, user roles 6, enumerations 3, time parsing 1) and 32 integration and system-level tests (workflow 13, access control 6, authentication 5, integrity 3, and administration, injection, audit trail, persistence and performance one each). Line coverage of the business-logic layer (domain and services) is **91.3%** (567 of 621 lines); data layer 79.8%. System tests cover the use case flows: a mark stays hidden until released, work 25 hours late is penalised 20%, work after three days scores zero, resubmission after the deadline or after marking is refused, an unenrolled student or non-teaching lecturer is refused, a HoD cannot moderate another department, and a returned mark cannot be revised without a reason. A lookup among 800 stored students completes well inside the 2-second target. Acceptance testing confirmed each stakeholder statement is met.

ID	Defect (severity)	Cause and fix	Retest
D1	Lecturer could be saved without a department (major)	Schema allowed null; added role CHECK	Pass
D2	Ambiguous integer binding failed to compile (minor)	Added `int` overload	Pass
D3	Results header missing an include (minor)	Added include	Pass
D4	29% coverage of enumeration conversions (minor)	Added tests for all conversions	90%
D5	Test helper namespace clashed with GoogleTest (minor)	Renamed namespace	Pass

Table 6: Defect log.

8. Configuration Management and Quality Assurance

Code, SQL, tests and documents are in one Git repository. The `main` branch always builds and passes tests; each requirement or fix is developed on a feature branch and merged by pull request after CI succeeds; commits are single-purpose with imperative messages; releases are tagged `v1.0` and `v1.1`. A GitHub Actions workflow runs on every push: it builds with warnings as errors, runs all tests with CTest, produces a gcovr coverage report for the domain and service layers and runs cppcheck. Code metrics: about 2,700 lines of core code, 745 of tests; estimated cyclomatic complexity is 3 for the penalty strategy, 7 for `ModerationService::review`, 12 for `SubmissionService::submit` and 13 for `MarkingService::saveMark`, all below my review threshold of 15. A self-review checklist led to concrete changes: a common exception hierarchy caught at the console boundary, an `IPasswordHasher` interface to keep libsodium out of the domain, a decoy hash against username enumeration, database CHECK constraints, a unit of work for atomic updates, and length limits on free-text input.

9. Deployment

The application is installed on one school computer holding the executable, `lms.db`, dated backups and submitted files; other workstations use it through remote console sessions on the school LAN. Sharing the SQLite file over a network drive was rejected because file locking is unreliable there. The deployment diagram is in the appendices file. *Installation:* install a C++17 compiler, CMake, SQLite and libsodium (on Ubuntu `sudo apt install cmake g++ pkg-config libsqlite3-dev libsodium-dev`); unpack release v1.0; run `cmake -S . -B build`, `cmake --build build` and `ctest --test-dir build`; copy `lms_app` to a data folder and start it; on first start create the initial administrator (no default account exists). Back up `lms.db` daily while the program is closed and keep two weeks of copies. User manuals for all roles and the v1.0 release notes (all of FR-01 to FR-19 and NFR-01 to NFR-07) are in the appendices file.

10. Maintenance: Change Request CR1

CR1: a lecturer can extend the deadline for a whole class, and extensions appear on the student's assignment list. This is **perfective maintenance** (new capability, with a small adaptive element in the deadline display) and activates FR-20. Two designs were compared: a nullable `student_id` meaning "whole class" (compact, but it needs a migration of a live

database and complicates every deadline query) and one approved extension row per enrolled student, created in one transaction. I chose the second because it needs no schema change, reuses the tested `DeadlinePolicy` and keeps a per-student audit trail.

Impact: requirements FR-07, FR-16, FR-20 and NFR-04; the extension use case and the assignment list; the extension sequence diagram; classes `AssignmentService` (new `grantClassExtension`), `ConsoleApp` (menu and list display) and the existing enrolled-students query; table `extensions` (new rows only); extension and display tests; user manuals and release notes. Implemented as v1.1: the lecturer must teach the course, give a reason and a date later than every enrolled student's deadline; all extensions are added atomically with one audit entry; the student list shows the effective deadline. **Regression:** the 61 v1.0 tests were re-run unchanged and passed before the new tests (all enrolled students extended, others unaffected, earlier dates rejected, rollback on failure, non-teaching lecturer refused, list shows the extended date) were accepted. Workflow and test run for v1.1 attached here (view run #59): https://github.com/Kirito-36/Learning-Management-System/actions

11. Professional, Legal and Ethical Issues

Data protection. The system holds personal information about learners, many of them children, so Lesotho's Data Protection Act, 2011 (Act No. 5 of 2012) applies. Its principles require specified and legitimate purposes, adequate and not excessive data, protection against unauthorised access or loss, and limited retention. Reports indicate the Data Protection Commission it provides for is not yet established, so I treated the Act's principles as the standard regardless. The design stores only what the case requires (no birth dates, addresses or photographs), uses data only for teaching and assessment, hashes credentials, uses parameterised SQL, limits each user to their own data (a HoD to their department) and leaves retention to the school, which also remains responsible for lawful grounds such as parental consent. **Assessment records.** Moderated release, the append-only audit log, transactions and backups protect integrity; physical access and careless administration need procedures (locked rooms, individual accounts, prompt deactivation of leavers). **Accessibility.** A keyboard-only text interface avoids reliance on colour or pointing and works with screen readers, but it has not been tested with assistive technology, and WCAG 2.2 (W3C, 2023) does not directly govern console programs; a future graphical interface should meet it from the start. **Fairness.** The late-penalty rule is transparent (students are told the penalty on submission), consistent (one strategy for everyone) and proportionate for short delays. It is harsh in two ways: zero after three days ignores effort, and a day starts the second the deadline passes. Because unreliable internet may cause lateness that is not the student's fault, the design accepts late work, allows audited extensions (class-wide from v1.1) and records every mark change to support appeals; the school should review the rule. I followed the ACM Code of Ethics (ACM, 2018), respected open-source licences and used only synthetic test data.

12. Conclusion

The project delivered a working Moderated LMS with consistent artefacts from requirements to maintenance. The decisions that mattered most were resolving stakeholder conflicts before design, separating marking, approval and release, isolating the penalty rule as a Strategy, confining SQL to a repository layer, hashing passwords with Argon2id and keeping an append-only audit log. Boundary-focused V-Model testing (61 tests, 91.3% business-logic coverage) and the low-impact CR1 change showed the design works. Limitations are the single-computer console interface, files referenced by path rather than stored, no student-initiated extension requests and untested accessibility. Future work could add request-and-approve extensions, an archive and deletion tool, in-application file storage and a graphical or web interface.

13. References

ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: ACM. | Biryukov, A., Dinu, D., Khovratovich, D. and Josefsson, S. (2021) *Argon2 Memory-Hard Function for Password Hashing*, RFC 9106. IETF. | Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley. | Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns*. Reading, MA: Addison-Wesley. | ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Requirements engineering*. Geneva: ISO. | Kingdom of Lesotho (2012) *Data Protection Act, 2011 (Act No. 5 of 2012)*, Government Gazette Vol. 57 No. 19. | Myers, G.J., Sandler, C. and Badgett, T. (2011) *The Art of Software Testing*. 3rd edn. Hoboken: Wiley. | OWASP (2024) *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org (Accessed: 3 November 2026). | Royce, W.W. (1970) 'Managing the development of large software systems', *Proc. IEEE WESCON*, pp. 1-9. | Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson. | W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C.

14. Appendices

The traceability matrix, full test results, user manuals (all roles), final risk register, use case diagram, UC-02 and UC-03 specifications, activity and sequence diagrams, ERD, deployment diagram, release notes and SQL schema are provided in the separate file *LMS_Report_Appendices*.